PukSquash Developer Manual

Project Setup and Firebase Data Layer

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

PukSquash lets NWU Potchefstroom students check squash court availability and manage bookings remotely. This part explains the environment shared by the functionality manuals. The application uses Vue 3 and Vite for the browser, Laravel 12 and PHP for the REST API, Firebase Authentication for identity, and Firebase Data Connect with Cloud SQL PostgreSQL for relational data.

The browser sends a Firebase ID token to Laravel. Middleware verifies the identity, controllers apply feature rules, repositories call named GraphQL operations, and SqlConnectClient sends those operations to Data Connect. DTO classes translate the GraphQL response into the API field names consumed by Vue. Use revision fe3e361 when reproducing this edition; later repository code uses a different persistence implementation.

1 1 Files and Responsibilities

resources/js/pages and components — application screens and reusable interface elements

resources/js/services/http.js — authenticated JSON requests

app/Http/Controllers/Api and app/Repositories — feature logic and data access

app/Services/SqlConnectClient.php — Data Connect transport

dataconnect/schema/schema.gql — six core entities

dataconnect/example/queries.gql and mutations.gql — named operations

1 2 API Contract

All feature URLs in these manuals begin with /api. GET /api/me is the initial identity and profile check. Protected requests use Authorization: Bearer <Firebase ID token>.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Local development setup

Windows commands to run from the Firebase baseline checkout

```
composer install
copy .env.example .env
php artisan key:generate
npm ci
php artisan serve
# In a second terminal
npm run dev
```

Explanation

Use PHP 8.2 or a compatible version accepted by composer.lock, Composer, and a Node version supported by the locked Vite release. Install from the existing lockfiles. Copy the environment template only for a new local setup. Laravel serves the application; Vite provides development assets. Create the local Laravel framework database if the chosen cache/session drivers need it. Laravel framework migrations do not create the Data Connect domain schema.

Physical Code Explanation

1 composer install resolves the PHP libraries, including kreait/firebase-php, from composer.lock.

2 key:generate creates the Laravel application key. npm ci installs the JavaScript lockfile without selecting arbitrary newer versions.

3 php artisan serve starts the PHP application, while npm run dev starts Vite. npm run build produces production assets; neither command deploys the backend.

Set the six VITE_FIREBASE_* names read by resources/js/services/firebase.js. On the server, set FIREBASE_PROJECT_ID, FIREBASE_CREDENTIALS, FIREBASE_DATACONNECT_LOCATION, FIREBASE_DATACONNECT_SERVICE and FIREBASE_DATACONNECT_CONNECTOR. Credential paths are resolved relative to the repository by this implementation. Keep the service-account JSON outside the public directory and source control.

3 Data Connect request implementation

app/Services/SqlConnectClient.php | source line 69 onward; blank and comment lines omitted

```
public function execute(string $operationName, array $variables = []): array
{
    $query = $this->loadOperation($operationName);
    $request = Http::timeout(15);
    if (! $this->emulatorHost) {
        $request = $request->withToken($this->getAccessToken());
    }
    $response = $request->post($this->endpoint, [
        'query'         => $query,
        'operationName' => $operationName,
        'variables'     => (object) $variables,
    ]);
    if ($response->failed()) {
        throw new RuntimeException(
            "SQL Connect request [{$operationName}] failed: " . $response->body(),
            $response->status(),
        );
    }
    $body = $response->json();
```

Explanation

Each repository selects an operation by its exact GraphQL name and supplies a variables array. The service loads the definition from the connector files. A server OAuth access token authorises the request in the hosted environment; DATA_CONNECT_EMULATOR_HOST selects the local emulator and skips that token path.

Physical Code Explanation

1 loadOperation locates the named query or mutation and caches its text. A missing operation name throws an exception.

2 Http::timeout(15) bounds the external request. withToken supplies the server access token; it is separate from the browser Firebase ID token.

3 The request posts query, operationName and variables. An empty variables array is cast to an object so GraphQL receives the expected JSON shape.

User is linked to Firebase through firebaseUid. Court and TimeSlot describe bookable cells; Booking reserves a court/date/slot; BookingPlayer records participants; CoachAvailability has a composite coach/slot/date key. Connector operations use NO_ACCESS for privileged server access. Firestore collections and Storage rules from the example manual do not describe this relational data model.

4 Verification and Maintenance

Verification checklist

1 Run npm run build and php artisan test in the configured Firebase checkout. Record the actual output; these manuals do not claim a test run has passed.

2 Confirm GET /api/me works with a valid token and returns 401 without one. Confirm a Data Connect query reaches the intended project and service.

3 Use the Firebase CLI and the repository dataconnect configuration to prepare the emulator or deploy the schema and connector in a development project before seeding courts and slots. Review schema changes before deployment.

4 Check every one-hour TimeSlot and court before testing bookings. Keep production credentials and production data out of local tests.

Implementation boundaries

The checked firebase.json configures Data Connect and its emulator. It has no complete hosting configuration for the Laravel PHP backend. Provision a PHP-capable HTTPS host and deploy Vite assets with Laravel; do not assume firebase deploy hosts the whole application.

The specifications require transaction-safe reservations, booking limits and additional Phase 2 tools. Comments about uniqueness are not proof that a PostgreSQL constraint exists. Validate deployed constraints and add integration tests. Time-slot administration and configurable rules have no complete admin API in this baseline.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).

https://firebase.google.com/docs/sql-connect/admin-sdk